

Online Learning of Modular Bayesian Deep Receivers: Single-Step Adaptation with Streaming Data

Yakov Gusakov, Osvaldo Simeone, Tirza Routtenberg, and Nir Shlezinger

Abstract—Deep neural network (DNN)-based receivers offer a powerful alternative to classical model-based designs for wireless communication, especially in complex and nonlinear propagation environments. However, their adoption is challenged by the rapid variability of wireless channels, which makes pre-trained static DNN-based receivers ineffective, and by the latency and computational burden of online stochastic gradient descent (SGD)-based learning. In this work, we propose an online learning framework that enables rapid low-complexity adaptation of DNN-based receivers. Our approach is based on two main tenets. First, we cast online learning as Bayesian tracking in parameter space, enabling a single-step adaptation, which deviates from multi-epoch SGD. Second, we focus on modular DNN architectures that enable parallel, online, and localized variational Bayesian updates. Simulations with practical communication channels demonstrate that our proposed online learning framework can maintain a low error rate with markedly reduced update latency and increased robustness to channel dynamics as compared to traditional gradient descent based method.

I. INTRODUCTION

A. Context and Motivation

The increasing demand for wireless data motivates the integration of emerging technologies such as massive and holographic multiple-input multiple-output (MIMO) [2] and reconfigurable intelligent surfaces [3]. While these innovations expand the capacity and functionality of next-generation networks, they also substantially complicate receiver design, challenging the applicability of classical model-based signal processing techniques [4], [5]. A promising direction to cope with such complexity is the incorporation of artificial intelligence (AI) tools, and particularly deep neural networks (DNNs), into the receiver chain [6]. By learning to infer from data, deep receivers offer a model-agnostic alternative that can operate effectively in unknown and nonlinear environments [7], [8].

Despite their promise, the deployment of deep receivers in practical wireless systems is subject to two fundamental constraints [9]: (i) wireless channels are inherently time-varying, implying that a suitable mapping learned by a DNN during training may quickly become outdated; and (ii) wireless devices typically operate under stringent constraints on computation,

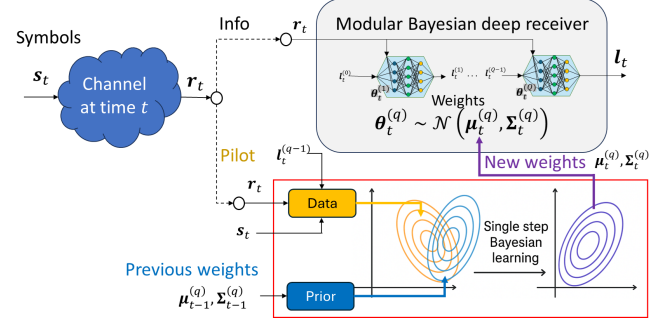


Fig. 1. Illustration of proposed framework for single-step online adaptation of modular Bayesian deep receivers

memory, and energy consumption. The first constraint motivates training deep receivers on-device, while the second constraint poses a challenge for standard deep learning algorithms, which are typically data- and compute-intensive [10].

Two main paradigms are considered for incorporating deep learning into the physical layer of wireless receivers. The first relies on offline pre-trained DNNs, inheriting a common framework in traditional deep learning domains such as computer vision and natural language processing. These models are typically trained over large datasets encompassing diverse channel realizations, aiming to learn mappings that generalize across multiple environments via *joint learning* [7], [11].

Some level of adaptivity can be achieved with pre-trained models using *ensemble-based architectures* [12], which maintain a set of specialized DNNs for different conditions; conventional linear channel estimates as additional inputs in inference [13], [14]; or hypernetworks that alter the DNN mapping based on an external DNN [15], [16]. Despite their simplicity in deployment, these approaches tend to produce highly parameterized models with elevated memory and compute requirements, limiting their feasibility for edge devices. Moreover, such pre-trained receivers struggle to generalize to unseen channel distributions without explicit adaptation, and often incur latency due to large inference pipelines [17].

An alternative paradigm to using pre-trained models employs *online learning*, wherein the deep receiver is adapted continuously using data from the current operating conditions [10]. This enables adaptation to channel variations, maximizing performance by tailoring the receiver to the instantaneous channel distribution. However, online training with conventional deep learning methods is often impractical in wireless devices due to computational constraints and the lack of abundant labeled data. Real-time updates require significant processing power and memory, while pilots offer limited

Parts of this work were presented at the 2025 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP) as the paper [1]. Y. Gusakov, T. Routtenberg, and N. Shlezinger are with the School of ECE, Ben-Gurion University of the Negev, Be'er-Sheva, Israel (e-mail: gusakovy@post.bgu.ac.il, {tirzar; nirshl}@bgu.ac.il). O. Simeone is with the Institute for Intelligent Networked Systems, Northeastern University London, One Portoken Street, London, E1 8PH, UK (email: o.simeone@northeastern.edu). This work was supported by the European Research Council (ERC) under the European Union's Horizon Europe Programme under the ERC starting grant nr. 101163973 (FLAIR) and by the Israel Science Foundation (grant No. 3314/25). The work of O. Simeone was supported by the ERC under the European Union's Horizon Europe Programme (grant agreement No. 101198347), by an Open Fellowship of the EPSRC (EP/W024101/1), and by the EPSRC project (EP/X011852/1).

samples that may be insufficient for effective training.

In this work, we introduce a novel online learning paradigm for deep learning-aided wireless receivers, illustrated in Fig. 1, which jointly leverages two complementary principles: (i) modular designs of the receiver architecture; and (ii) Bayesian DNNs. While modularity and Bayesian learning have each independently shown promise in facilitating efficient AI-aided receivers, we combine them into a unified framework for *rapid adaptation* inspired by recent advances in continual Bayesian learning [18]–[20]. The key conceptual insight of our approach is to cast the task of online training as the tracking of a dynamic system in the parameter space [21], where statistical observations stem from received pilot signals. This formulation enables the application of recursive Bayesian filtering methods to online learning, facilitating single-step updates in response to time-varying channel dynamics.

B. Related Work

The complexity of online adaptation can be partially tackled by using several complementary approaches. These include architectures that integrate domain knowledge to yield compact and modular designs that reduce training complexity [22]–[24]; meta-learning techniques for optimizing the hyperparameters of the learning algorithm [25], [26]; and the incorporation of change detection mechanisms [27] to identify when retraining is necessary.

To obtain training data for online adaptation, it was proposed to exploit the structure of communication protocols [28], [29] and modulation schemes [30] for self-supervision. This approach can be combined with targeted data augmentation [31], [32] to obtain labeled data. Moreover, Bayesian learning tools were proposed to avoid overfitting with limited data [33]–[35], and their improved calibration was shown to be highly effective when combined with modular deep architectures, enabling improved generalization and performance [36].

Yet, a central bottleneck persists: nearly all existing methods rely on stochastic gradient descent (SGD) or its variants, which necessitate multiple passes over data and slow convergence. These characteristics make them ill-suited for scenarios where channel conditions evolve rapidly and adaptation must occur within the order of a coherence block of a wireless channel. This gives rise to the need for novel online learning mechanisms that can perform effective updates, by potentially deviating from conventional and widely-accepted SGD-based learning.

C. Main Contributions

In this work, we propose a low-latency, high-performance online training algorithm for deep receivers based on Bayesian training and modular architectures. Our main contributions are summarized as follows:

- **Online learning as Bayesian tracking:** We cast the online training of deep receivers as a Bayesian tracking task over a dynamic system defined by the evolving DNN parameters via variational learning [37]. This enables the principled application of recursive Bayesian estimation tools, which are well-suited for low-latency adaptation in dynamic environments.
- **Natural-gradient-based online Bayesian learning:** We adapt online Bayesian learning algorithms grounded in

natural-gradient approximations to implement online learning via Bayesian tracking.

- **Complexity-aware streaming modular adaptation:** We integrate the proposed Bayesian learning techniques with modular deep receivers to support *module-wise* Bayesian updates, and treat channel outputs as *streaming data* to enable Bayesian learning in a pipelined and asynchronous manner, leading to scalable, fast, and localized online training with minimal overhead.
- **Extensive experimental validation:** We validate our proposed methodology through extensive simulations involving both synthetic and realistic wireless environments. In particular, we consider dynamic fading scenarios generated using the Sionna [38], COST2100 [39], and QuaDRiGa [40] simulators. The experiments demonstrate our ability to rapidly adapt to time-varying conditions while achieving state-of-the-art performance in terms of uncoded bit error rate (BER) and robustness to channel variations.

The rest of this paper is structured as follows. Section II presents the system model, comprised of both the channel model and receiver architecture. Section III describes our modular online Bayesian learning approach, which we evaluate in Section IV, while Section V provides concluding remarks.

D. Organization and Notation

Throughout this paper, we use boldface lower-case and upper-case letters for vectors (e.g., $\mathbf{x}$) and matrices (e.g., $\mathbf{X}$), respectively. The (i, j) th entry of $\mathbf{X}$ is denoted by $[\mathbf{X}]_{i,j}$. Calligraphic letters denote sets, e.g., $\mathcal{X}$, with $|\mathcal{X}|$ being the cardinality of $\mathcal{X}$. We use $\mathbb{R}$ and $\mathbb{C}$ for the sets of real and complex numbers, respectively. The notation $\mathbb{P}(\cdot)$ is the probability function, while $\mathbb{E}[\cdot]$ is the expectation operator. When a subscript is specified, e.g., $\mathbb{E}_{\theta \sim q}[\cdot]$, the expectation is taken with respect to the random variable θ distributed according to $q(\theta)$. We use $(\cdot)^\top$, $\odot$, and $\text{diag}(\cdot)$ to denote the transpose operator, the element-wise product of two vectors, and a diagonal matrix with the elements of a vector on its diagonal, respectively. We use $\Re\{\cdot\}$ and $\Im\{\cdot\}$ to denote the real and complex parts, respectively, of a scalar or vector. The gradient operator $\nabla_{\mathbf{z}} f(\mathbf{z})$ denotes a column vector of partial derivatives, i.e., $\nabla_{\mathbf{z}} f = [\frac{\partial f}{\partial z_1}, \dots, \frac{\partial f}{\partial z_P}]^\top$.

II. SYSTEM MODEL AND PRELIMINARIES

In this section, we introduce the system model, describing the time-varying channel model and transmission scheme in Subsection II-A, and the modular receiver architecture in Subsection II-B. In Subsection II-C, we formulate the online adaptation problem, and review key preliminaries in Subsection II-D. The main symbols defined in the following are summarized in Table I.

A. Time-Varying Channel Model

1) *Channel Model:* We consider an uplink MIMO system with K single-antenna users communicating with an N -antenna receiver. Each symbol $s_t^{(k)} \in \mathcal{S}$ transmitted from user k at time t is drawn from a constellation of size $|\mathcal{S}| = 2^B$, where each symbol modulates B bits. In particular, let

$$\mathbf{b}_t^{(k)} = [b_{t,1}^{(k)}, \dots, b_{t,B}^{(k)}]^\top \in \{0, 1\}^B, \quad k = 1, \dots, K, \quad (1)$$

TABLE I
SUMMARY OF SYMBOLS

Symbol	Description
t	Time index
K	Number of receiver users
N	Number of receiver antennas
$\mathcal{S} \subset \mathbb{C}$	Symbol constellation
$B = \log_2 \mathcal{S} $	Number of bits per symbol
$\mathbf{b}_t \in \{0, 1\}^{KB}$	Bit vector
$\mathbf{s}_t \in \mathcal{S}^K$	Transmitted symbol vector
$\mathbf{r}_t \in \mathbb{C}^N$	Received signal
$\ell_t \in [0, 1]^{KB}$	Soft bit-level estimates
$\mathbf{x}_t \in \mathbb{R}^{2N+KB}$	DeepSIC module input
P	Number of receiver parameters
$\boldsymbol{\theta}_t \in \mathbb{R}^P$	Parameters of the deep receiver

denote the binary vector, which is mapped to the symbol $s_t^{(k)}$. The bit vector transmitted by all K users at time t is denoted

$$\mathbf{b}_t = \left[\left(\mathbf{b}_t^{(1)} \right)^\top, \left(\mathbf{b}_t^{(2)} \right)^\top, \dots, \left(\mathbf{b}_t^{(K)} \right)^\top \right]^\top \in \{0, 1\}^{KB}, \quad (2)$$

and the corresponding complete symbol vector is defined as

$$\mathbf{s}_t = \left[s_t^{(1)}, \dots, s_t^{(K)} \right] \in \mathcal{S}^K. \quad (3)$$

The received signal, $\mathbf{r}_t \in \mathbb{C}^N$, is characterized by a generic time-varying conditional distribution,

$$\mathbb{P}(\mathbf{r}_t = \mathbf{r} | \mathbf{s}_t = \mathbf{s}), \quad (4)$$

representing dynamic channel conditions. Beyond this general input-output relationship, we do not impose a specific channel model, allowing for complex and highly nonlinear channels. In the following, $\mathbb{P}(\cdot)$ denotes the true distribution governing the data as in (4), while $p(\cdot)$ denotes an induced or assumed probability measure.

2) *Temporal Variations*: A common approach in the MIMO literature adopts a block-fading assumption, where the channel is treated as approximately constant over a coherence interval and assumed to change independently between successive blocks [41]. While this model simplifies receiver design, it does not fully capture the nature of real-world wireless channels, which often exhibit continuous variations over time, leading to effects such as channel aging within an assumed coherence block [42]. In this work, we allow the channel to change within each block. It is only assumed that the conditional probability from (4) evolves smoothly over time.

3) *Transmission Scheme*: We consider a generic pilot-based transmission scheme as illustrated in Fig. 2. Here, transmission consists of two phases:

1. A *synchronization phase* during which T_{sync} predetermined symbols $\{\mathbf{s}_t\}_{t=1, \dots, T_{\text{sync}}}$ are transmitted. Such sequences, commonly employed by wireless protocols for transmission alignment, are used here for the initial calibration of our deep receiver.
2. A *tracking phase*, where data blocks are transmitted with pilot symbols periodically inserted between them. The receiver decodes the data symbols in real time, and uses the pilot symbols to fine-tune its operation, e.g., to adapt the deep receiver through online learning.

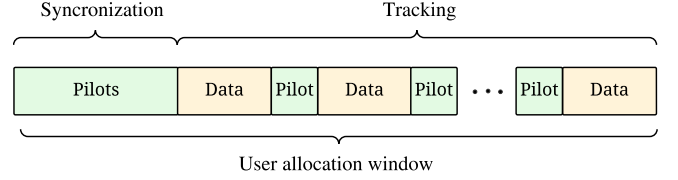


Fig. 2. Transmission scheme with synchronization and periodic pilot-data blocks.

B. Receiver Architecture

The demodulator is implemented as a DNN parameterized by a vector of weights $\boldsymbol{\theta}_t$, which can be adapted over time to track changing channel conditions. At each time step t , the DNN maps the received signal $\mathbf{r}_t \in \mathbb{C}^N$ to soft bit level estimates, denoted by

$$\ell_t = \mathbf{h}_{\boldsymbol{\theta}_t}(\mathbf{r}_t) \in [0, 1]^{KB}. \quad (5)$$

The output vector is structured as

$$\ell_t = \left[\ell_{t,1}^{(1)}, \dots, \ell_{t,B}^{(1)}, \dots, \ell_{t,1}^{(K)}, \dots, \ell_{t,B}^{(K)} \right]^\top, \quad (6)$$

where $\ell_{t,i}^{(k)}$ is the soft estimate of the i -th bit of user k at time t . It can be interpreted as the estimated conditional probability that $b_{t,i}^{(k)} = 1$ conditioned on the received signal and the DNN parameters, i.e.,

$$\ell_{t,i}^{(k)} = p(b_{t,i}^{(k)} = 1 | \mathbf{r}_t, \boldsymbol{\theta}_t). \quad (7)$$

For uncoded transmission, hard decisions on transmitted bits are made by thresholding the soft estimates, i.e.,

$$\hat{\mathbf{b}}_t = \mathbb{1}[\ell_t > 0.5] \in \{0, 1\}^{KB}, \quad (8)$$

where the comparison and the indicator function $\mathbb{1}[\cdot]$ are applied element-wise.

C. Problem Formulation

Our goal is to develop an online learning algorithm for enabling adaptive and flexible deep receivers by optimizing the DNN parameters based on the incoming pilot symbols. Formally, after receiving the channel output $\mathbf{r}_t$ corresponding to a pilot symbol $\mathbf{s}_t$, the algorithm maps them to an updated parameter vector $\boldsymbol{\theta}_t$. The receiver is then used to detect the subsequent data symbols. The objective is to minimize the BER in the data-bearing portion of the transmission.

Let $\mathcal{T}_{\text{data}}$ denote the set of time indices corresponding to the data symbols. The optimization problem can be expressed as:

$$\min_{\{\boldsymbol{\theta}_t\}} \frac{1}{BK|\mathcal{T}_{\text{data}}|} \sum_{t \in \mathcal{T}_{\text{data}}} \sum_{k=1}^K \sum_{i=1}^B \mathbb{P}(\hat{b}_{t,i}^{(k)} \neq b_{t,i}^{(k)} | \mathbf{r}_t, \boldsymbol{\theta}_t), \quad (9)$$

where the decoded bits $\hat{\mathbf{b}}_t$ depend on $\boldsymbol{\theta}_t$ through the neural network mapping in (5) and (8).

This optimization of the DNN is to be carried out sequentially in an *online* manner, updating $\boldsymbol{\theta}_t$ using only the information available up to time t . The online learning algorithm must be capable of tracking a smoothly time-varying, potentially nonlinear channel, while adhering to the practical constraints of wireless communication systems, namely, non-stationary signal statistics, and low-latency requirements. The key challenges include:

- C1 **Non-stationarity:** The channel distribution evolves rapidly over time, requiring continuous adaptation of the receiver.
- C2 **Latency and complexity:** Receiver updates must be computationally efficient and meet strict runtime budgets for real-time operation.
- C3 **Nonlinearity:** The channel (4) at each time t may be highly nonlinear, difficult to parametrize explicitly, and possibly include non-Gaussian stochasticity. This makes classical model-based adaptation impractical.

D. Preliminaries

1) *Bayesian Deep Learning:* Bayesian deep learning offers a framework to quantify uncertainty in DNN predictions by treating the DNN parameters θ as random variables with prior $p(\theta)$ and learning their posterior distribution $p(\theta|\mathcal{D})$ given the data $\mathcal{D}$ [33], [43]. The posterior can be represented explicitly, as in variational inference [44], or implicitly via alternative induced trainable stochasticity, as in Monte Carlo dropout [45].

Specifically, in standard (offline) variational inference, the posterior $p(\theta|\mathcal{D})$ is explicitly approximated by a *variational distribution* $q_\psi(\theta)$, parametrized by the *variational parameters* ψ . These parameters are learned by the minimization of the evidence lower bound (ELBO) objective, which consists of the expected negative log-likelihood regularized by a Kullback–Leibler divergence (KLD) term between the induced $q_\psi(\theta)$ and the prior $p(\theta)$. For a labeled dataset of the form $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{|\mathcal{D}|}$, this loss is given by

$$\mathcal{L}_{\mathcal{D}}^{\text{ELBO}}(\psi; p(\theta)) = -\frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \mathbb{E}_{\theta \sim q_\psi} [\log p(\mathbf{y}_i | \mathbf{x}_i, \theta)] + D_{\text{KL}}(q_\psi(\theta) || p(\theta)), \quad (10)$$

where D_{KL} denotes the KLD. The ELBO enables gradient-based learning using, e.g., Bayes-by-backprop (BBB) [46].

During inference, J realizations $\{\theta_j\}_{j=1}^J$ are drawn in an i.i.d. manner from the distribution q_ψ to obtain an ensemble $\{p(\mathbf{y} | \mathbf{x}, \theta_j)\}_{j=1}^J$ whose predictive distributions are averaged to obtain a final prediction. Compared to standard (frequentist) DNNs, Bayesian neural networks (BNNs) were shown to be less prone to overfitting, and their output probabilistic estimates are typically more calibrated, i.e., more faithful to the actual prediction uncertainty [47].

2) *Modular Deep Receivers:* Modular deep receivers are deep learning-based architectures that incorporate task-specific structure by associating sub-networks with identifiable sub-tasks in the receiver processing chain. This design enables the use of compact DNN modules with interpretable output features, localized parameter updates, and improved scalability [10]. In particular, such architectures naturally arise from model-based deep learning methods [48], where the algorithmic steps of classical receivers are integrated into the network structure via unfolding or architectural priors.

Our main example used in this paper of a modular deep receiver is *DeepSIC* [23], which unfolds the iterations of the soft interference cancellation (SIC) algorithm of [49] for uplink MIMO detection into a modular DNN architecture. As illustrated in Fig. 3, DeepSIC consists of Q iterations, each

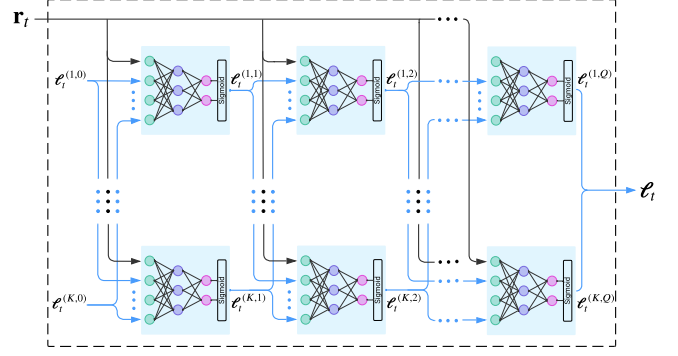


Fig. 3. DeepSIC model architecture illustration.

with K separate modules. The k th module at iteration q is responsible for decoding the symbol of user k while canceling the interference from the remaining users. The parameter vector of the receiver at time t , denoted θ_t , can be decomposed as $\theta_t = \{\theta_t^{(k,q)}\}_{k,q=1}^{K,Q}$, where $\theta_t^{(k,q)}$ denotes the weights of the k th user's module at the q th SIC iteration.

Specifically, the k th module at iteration q produces a vector of soft estimates, denoted by $\ell_t^{(k,q)}$, by applying the user-specific DNN mapping $\mathbf{h}_{\theta_t^{(k,q)}}$ to an input vector shared by all modules in the layer, which is comprised of the received signals and the previous iteration's output as

$$\mathbf{x}_t^{(q)} = \left[\Re\{\mathbf{r}_t^\top\}, \Im\{\mathbf{r}_t^\top\}, \left(\ell_t^{(q-1)}\right)^\top \right]^\top, \quad (11)$$

where

$$\ell_t^{(q)} = \left[\left(\ell_t^{(1,q)}\right)^\top, \dots, \left(\ell_t^{(K,q)}\right)^\top \right]^\top. \quad (12)$$

Consequently, the soft estimates are updated as

$$\ell_t^{(k,q)} = \mathbf{h}_{\theta_t^{(k,q)}}(\mathbf{x}_t^{(q)}). \quad (13)$$

The initial soft estimates are set to $\ell_t^{(k,0)} = [0.5, \dots, 0.5]^\top$ for all $k = 1, \dots, K$. The final soft estimates ℓ_t are taken from $\{\ell_t^{(k,Q)}\}_{k=1}^K$.

The resulting deep unfolded version of SIC enables accurate multi-user detection in complex nonlinear channels with non-Gaussian noise [23], thus handling Challenge C3. Moreover, the modular nature of DeepSIC was shown in [36] to be beneficial when combined with BNNs for its internal modules, as the improved calibration of Bayesian deep learning leads to more accurate soft estimates exchanged by the modules, which in turn leads to improved performance.

However, coping with nonstationarity (Challenge C1) requires the receiver to adapt its parameters θ_t (or their distribution parameters ψ_t for BNNs). To-date, online learning is still a complex and lengthy procedure (thus violating Challenge C2), as regardless of whether the architecture is modular or black-box and whether one employs BNNs or frequentist DNNs, training is still carried out using numerous sequential gradient updates based on SGD or its variants. Still, in our current context of online learning, modular architectures, such as DeepSIC, offer several advantages: (i) updates can be localized to specific modules corresponding

to users with significant channel variation; (ii) inference and training can be parallelized across modules; and (iii) the architecture naturally supports a pipelined processing strategy. In our proposed methodology, we exploit these properties, combined with BNNs, to enable efficient and scalable online Bayesian learning via module-wise updates, as detailed in Section III.

III. MODULAR ONLINE BAYESIAN LEARNING

This section introduces our proposed methodology for rapid adaptation based on modular online Bayesian learning. The goal is to track time-varying channels (Challenge C1) using deep receivers capable of handling complex nonlinearities (Challenge C3), while meeting stringent latency and complexity constraints (Challenge C2). In Subsection III-A we cast online training of BNNs as a *tracking* problem in a parameter-space dynamical system, where the received pilots serve as streaming measurements. In Subsection III-B we leverage this state-space representation to perform online learning via efficient recursive Bayesian filtering. This Bayesian framework, which integrates a level of uncertainty in the current model, enables replacing multiple gradient-descent iterations with a *single step* parameter update. Then, in Subsection III-C we combine Bayesian tracking with modular receiver architectures, and show how modularity facilitates low-complexity, low-latency online adaptation. We conclude by providing a discussion in Subsection III-D.

A. State Space Formulation

Our approach is based on modeling the online learning of deep receivers as a *dynamic system*. To formulate this, we consider a generic DNN-based receiver, which at time t is tasked with mapping its input $\mathbf{x}_t$ into bit-wise soft estimates, $\ell_t = h_{\hat{\theta}_t}(\mathbf{x}_t) \in (0, 1)^B$, aiming to recover the true bits (labels) denoted $\mathbf{b}_t \in \{0, 1\}^B$. A special case of this generic model is a single DeepSIC module that can be obtained by omitting the superscript k, q .

We model channel variations in the DNN parameter space via the *random process*, $\{\hat{\theta}_t\}_{t=1,2,\dots}$, where $\hat{\theta}_t$ denotes the desired DNN parameters at time t . Considering relatively smooth variations in the desired DNN parameters, we assume a first-order Gaussian Markov model of the form

$$p(\hat{\theta}_t | \hat{\theta}_{t-1} = \theta) = \mathcal{N}(\gamma\theta, \sigma^2 \mathbf{I}), \quad (14)$$

where parameter $\sigma^2 > 0$ controls the evolution rate, while parameter $\gamma \in (0, 1]$ determines the memory of the process. Values near 1 retain more past information, while smaller values enable faster adaptation to rapid changes. The initial weights are assumed to obey a Gaussian prior $p(\hat{\theta}_0) = \mathcal{N}(\mu_0, \Sigma_0)$.

At time t , the output of the DNN receiver with the desired parameters $\hat{\theta}_t$, given by $\ell_t = h_{\hat{\theta}_t}(\mathbf{x}_t)$, dictates the distribution of the vector of independent Bernoulli variables, such that

$$p(b_{t,i} = 1 | \mathbf{x}_t, \hat{\theta}_t) = \ell_{t,i}. \quad (15)$$

Consequently, the mean and covariance of the bit vector $\mathbf{b}_t$ conditioned on the desired DNN parameters $\hat{\theta}_t$ are given by

$$\mathbb{E}[\mathbf{b}_t | \mathbf{x}_t, \hat{\theta}_t] = \mathbf{h}_{\hat{\theta}_t}(\mathbf{x}_t), \quad (16)$$

$$\text{Cov}[\mathbf{b}_t | \mathbf{x}_t, \hat{\theta}_t] = \text{diag}(\mathbf{h}_{\hat{\theta}_t}(\mathbf{x}_t) \odot (\mathbf{1} - \mathbf{h}_{\hat{\theta}_t}(\mathbf{x}_t))). \quad (17)$$

Equations (14)-(15) define a *nonlinear state space model (SSM)* with *Gaussian state evolution*, where the latent states are the desired DNN parameters $\hat{\theta}_t \in \mathbb{R}^P$.

B. Bayesian Online Learning

We model online learning as the sequential estimation of desired DNN parameters $\hat{\theta}_t$ evolving as a Markov process (14) and generating noisy observations (15) of the information bits. Consequently, the online learning task is to process data points (pilots) that arrive sequentially and continuously update the belief about $\hat{\theta}_t$ as new pilot data $(\mathbf{x}_t, \mathbf{b}_t)$ arrives. To leverage this SSM for learning purposes, we adopt a *Bayesian deep learning* perspective, and model the receiver as a BNN whose parameters obey a Gaussian distribution (or a subfamily of Gaussian such as diagonal Gaussian). The variational distribution at time t , denoted by $q_{\psi_t}(\theta_t)$, is parameterized by a mean vector μ_t and a covariance matrix Σ_t , i.e.,

$$q_{\psi_t}(\hat{\theta}_t) = \mathcal{N}(\mu_t, \Sigma_t), \quad (18)$$

which serves as a tractable approximation for the true posterior $p(\hat{\theta}_t | \{(\mathbf{x}_\tau, \mathbf{b}_\tau)\}_{\tau=1}^t)$.

Casting the learning dynamics as a SSM and the learning task as updating a Gaussian belief gives rise to two online learning algorithms: (i) an algorithm based on a variant of the extended Kalman filter (EKF) [50], and (ii) an algorithm based on combining Bayesian learning and natural gradient updates [19]. Both approaches operate in a *one-shot* manner, allowing one to update with a *single gradient step* on a single data point.

1) *Conditional-Moments EKF*: The SSM representation in (14)-(15) suggests that online learning can be treated as the tracking of a latent state vector $\hat{\theta}_t$. Together with the imposed Gaussianity (18), this naturally motivates the use of Kalman-type algorithms to recursively update the posterior. Our primary learning method thus employs the conditional-moments EKF (CM-EKF) algorithm [50], which extends the EKF algorithm to non-Gaussian SSMs. The CM-EKF follows the standard *predict-update* structure of the EKF.

In the predict step, an approximate prior at time t is obtained by propagating the previous posterior from time $t-1$ through the Gaussian state evolution model in (14). Given the previous posterior $q_{\psi_{t-1}}(\hat{\theta}_{t-1})$, the predictive prior at time t is given by:

$$q_{\psi_{t|t-1}}(\hat{\theta}_t) = \int p(\hat{\theta}_t | \hat{\theta}_{t-1}) q_{\psi_{t-1}}(\hat{\theta}_{t-1}) d\hat{\theta}_{t-1}. \quad (19)$$

According to (18), we assume that we have a Gaussian posterior, $q_{\psi_{t-1}}(\hat{\theta}_{t-1}) = \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$. Thus, the integral in (19) yields another Gaussian distribution [51] with mean and covariance, respectively, obtained as

$$\mu_{t|t-1} = \gamma\mu_{t-1}, \quad (20)$$

$$\Sigma_{t|t-1} = \gamma^2 \Sigma_{t-1} + \sigma^2 \mathbf{I}. \quad (21)$$

In the update step, the predictive distribution is then used as a prior. Since the likelihood in (15) is a function of the network mapping, which is highly nonlinear, we employ a local linearization of the DNN mapping that transforms (16) into

an affine transformation of the state $\hat{\theta}_t$ by taking its first-order Taylor series approximation around (20). Namely, we have

$$\bar{h}_{\hat{\theta}_t}(x_t) = h_{\mu_{t|t-1}}(x_t) + H_t (\hat{\theta}_t - \mu_{t|t-1}), \quad (22)$$

where

$$H_t = \nabla_{\hat{\theta}}^\top h_{\hat{\theta}}(x_t) \Big|_{\hat{\theta}=\mu_{t|t-1}} \in \mathbb{R}^{KB \times P}$$

is the Jacobian of the network with respect to its weights.

The variational posterior at time t is obtained by the CM-EKF update rule, which for our setting of online learning maps the DNN input x_t and the corresponding pilot bits b_t into the BNN parameters ψ_t via

$$\mu_t = \mu_{t|t-1} + K_t (b_t - h_{\mu_{t|t-1}}(x_t)), \quad (23)$$

$$\Sigma_t = \Sigma_{t|t-1} - K_t H_t \Sigma_{t|t-1}. \quad (24)$$

Here K_t is the Kalman gain matrix that is given by

$$K_t = \Sigma_{t|t-1} H_t^\top (H_t \Sigma_{t|t-1} H_t^\top + R_t)^{-1}, \quad (25)$$

where

$$R_t \triangleq \text{Cov} [b_t | x_t, \hat{\theta}_t = \mu_{t|t-1}] \quad (26)$$

is computed according to (17) by setting $\hat{\theta}_t = \mu_{t|t-1}$. To ensure numerical stability, the diagonal entries of R_t are clipped from below at a minimum value, which we set to 0.1 in our numerical study.

Online learning via the CM-EKF updates the BNN distribution parameters (μ_t, Σ_t) in a single step. That is, each new observation x_t is processed once by the network (to compute $h_{\mu_{t|t-1}}(x_t)$), and only a single backward pass is required (to compute H_t). This stands in contrast to standard SGD-based learning, which typically requires numerous gradient steps. Moreover, CM-EKF does not require sampling from the variational distribution, making it a robust deterministic learning rule.

While the complexity of each SGD iteration only scales linearly with the number of DNN parameters P , the complexity of processing one observation vector with the CM-EKF is $\mathcal{O}(BP^2 + B^3)$ where BP^2 is typically the dominant term, as $P \gg B$, i.e., the number of DNN weights notably exceeds the number of bits extracted from x_t .

2) *Bayesian Online Natural Gradient*: CM-EKF is essentially an algorithm for the tracking of a dynamic system, where the aim is to have the posterior mean estimate the latent state in the mean squared error (MSE) sense [21]. As the goal in online learning is not necessarily to minimize the MSE with respect to the assumed desired parameters, but rather to minimize some learning loss function, as in e.g., the ELBO (10), one can formulate more conventional online learning algorithms that are derived from such loss formulations.

Recent advances in Bayesian learning from streaming data have developed methods that enable single-step online learning based on the ELBO. In particular, the Bayesian online natural gradient (BONG) algorithm proposed in [19] updates ψ_t based on the ELBO for the current sample (x_t, b_t) by taking a single natural gradient step while using $\mu_{t|t-1}$ as a starting point and

$q_{\psi_{t|t-1}}$ as a prior (so that the KL term vanishes when evaluated at $\psi = \psi_{t|t-1}$). The resulting online learning rule is given by

$$\psi_t = \psi_{t|t-1} + F_{\psi_{t|t-1}}^{-1} \nabla_{\psi_{t|t-1}} \mathbb{E}_{\theta \sim q_{\psi_{t|t-1}}} [\log p(b_t | x_t, \theta)], \quad (27)$$

where the Fisher information matrix (FIM) associated with the variational parameters $\psi_{t|t-1}$, is defined as

$$F_{\psi_{t|t-1}} = \mathbb{E}_{\theta \sim q_{\psi_{t|t-1}}} \left[\nabla_{\psi_{t|t-1}} \log q_{\psi_{t|t-1}}(\theta) \nabla_{\psi_{t|t-1}}^\top \log q_{\psi_{t|t-1}}(\theta) \right]. \quad (28)$$

When employing a Gaussian variational distribution where the BNN parameters are the natural parameters $\psi_t = (\Sigma_t^{-1} \mu_t, -\frac{1}{2} \Sigma_t^{-1})$, then (27) reduces to two update equations [19], given by

$$\mu_t = \mu_{t|t-1} + \Sigma_t \mathbb{E}_{\theta_t \sim \psi_{t|t-1}} [\nabla_{\theta_t} \log p(b_t | h_{\theta_t}(x_t))], \quad (29)$$

$$\Sigma_t^{-1} = \Sigma_{t|t-1}^{-1} - \mathbb{E}_{\theta_t \sim \psi_{t|t-1}} [\nabla_{\theta_t}^2 \log p(b_t | h_{\theta_t}(x_t))]. \quad (30)$$

The BONG update rule (29)-(30) requires computing stochastic expectations. To translate it into an online learning method, we consider three different forms of approximations:

A1 **Linearized Gaussian** approximations compute the expectations by adopting the linearized form (22) and imposing a Gaussian distribution on the likelihood, i.e.,

$$p(b_t | x_t, \hat{\theta}_t) \approx \mathcal{N}(\bar{h}_{\hat{\theta}_t}(x_t), R_t), \quad (31)$$

where R_t is defined in (26).

A2 **Empirical Fisher (EF)** replaces the Hessian in (30) with the outer-product of the sampled gradients [52].

A3 **Diagonal plus low-rank (DLR)** parameterization restricts the covariance in (29) to be of the form [53]

$$\Sigma_t^{-1} = D_t + W_t W_t^\top, \quad (32)$$

where D_t is a diagonal matrix and $W_t \in \mathbb{R}^{P \times R}$ with $R \ll P$.

Among the above options, EF approximations (A2) are typically less accurate and often lead to instability. When using a linearized Gaussian formulation (A1), the stochastic expectations exhibit a closed-form expression of the model parameters and the linearized model H_t . In fact, under this modeling, the BONG update in (29)-(30) coincides with the CM-EKF update equations (23)-(24) [19]. While the DLR parameterization (A3) does not allow computing the expectations on its own, it can be combined with A1 to avoid the quadratic complexity growth of the CM-EKF with the number of parameters P . Specifically, the BONG update under A1+A3 is termed *variational-diagonal extended Kalman filter (VD-EKF)* [18] when setting $R = 0$ in (32), or *Lo-Fi* [54] for $R > 0$, and in both cases the resulting complexity only grows linearly with P .

In summary, the BONG algorithm allows one-step online learning based on the ELBO objective. Although this step can be computationally heavy due to the need to evaluate stochastic expectations, it can be approximated with reduced complexity. The most stable and expressive approximation coincides with CM-EKF, the fastest coincides with VD-EKF, while Lo-Fi balances complexity and performance, as also shown in our numerical study in Section IV.

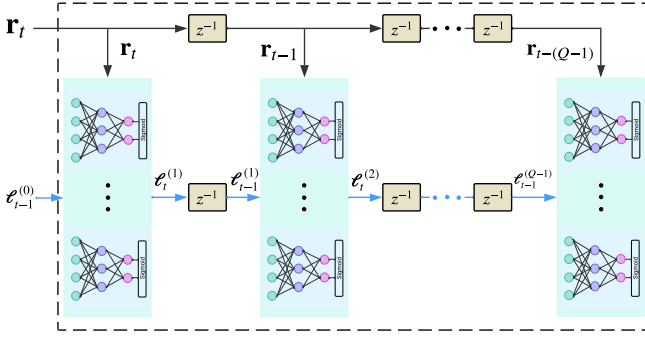


Fig. 4. Pipelined DeepSIC model architecture illustration.

C. Modular Bayesian Online Adaptation

The complexity associated with full covariance adaptation methods such as CM-EKF grows at least quadratically with the number of DNN weights P , notably limiting their usefulness for rapid adaptation of large DNNs. We next demonstrate how both complexity and latency can be alleviated by exploiting modular architectures such as DeepSIC. In the following, we build upon the state space formulation (Subsection III-A) and recursive online Bayesian learning algorithms (Subsection III-B), to show how modular architectures (focusing on DeepSIC) enable rapid online adaptation via module-wise updates, parallelization across user modules, and low-latency pipelining, harnessing the treatment of channel outputs as streaming data.

Module-Wise Updates: Each DeepSIC module, indexed by (k, q) , maintains its own Gaussian posterior $q_{\psi_t}^{(k,q)} = \mathcal{N}(\mu_t^{(k,q)}, \Sigma_t^{(k,q)})$. Since each module is tasked with producing a soft estimate of the k th user bits $\mathbf{b}_t^{(k)}$, we can assign a label for each module, and thus formulate a *separate module-wise SSM*. Accordingly, each module can be evaluated and trained separately via one-shot Bayesian online learning based on its corresponding SSM.

While BNNs typically come at the cost of slow inference due to the need to sample multiple i.i.d. realizations of $\theta_t^{(k,q)} \sim q_{\psi_t}^{(k,q)}$, here we employ the Bayesian modeling only for learning purposes. During inference, we plug in the mean parameter of the variational distribution, i.e., set $\theta_t^{(k,q)} = \mu_t^{(k,q)}$, to avoid the overhead and excessive latency of sampling.

Parallelization: The ability to disentangle the update of each module can notably facilitate online learning, particularly when combined with some form of parallel processing capabilities. Specifically, all K modules in iteration q share the same input $\mathbf{x}_t^{(q)}$ and can compute their update rules in parallel. In practice, this leverages either multicore or vectorization hardware to achieve up to K -fold speed-up over sequential processing.

Pipelining: Since DeepSIC iterates $q = 1, 2, \dots, Q$ sequentially, a conventional implementation would process each sample through all Q iterations before processing the next. In our case, we view the channel outputs as streaming data, where both inference and online training are done on each sample rather than on aggregated batches. This approach allows us to pipeline Q samples through the Q iterations, so that at each time step, iteration q processes sample $t - (q - 1)$ for online adaptation or inference, as illustrated in Fig. 4. This reduces overhead by up

to a Q -fold, introducing minimal latency: at time t , the network outputs soft estimates of bits at $t - Q + 1$. Such latency is often acceptable, particularly since digital communications typically employ subsequent channel coding, where a sequence of soft estimates are gathered at the output of the demodulator before the decoded bits are extracted, and thus there is some tolerable latency before data is decoded and processed.

Algorithm Summary: Upon receiving pilot $(\mathbf{r}_t, \mathbf{b}_t)$, each module applies predict and update steps using $(\hat{\mathbf{x}}_t^{(q)}, \mathbf{b}_{t-(q-1)}^{(k)})$, where

$$\hat{\mathbf{x}}_t^{(q)} = \left[\Re \{ \mathbf{r}_{t-(q-1)}^\top \}, \Im \{ \mathbf{r}_{t-(q-1)}^\top \}, \left(\ell_{t-1}^{(q-1)} \right)^\top \right]^\top. \quad (33)$$

The update step can be based on the CM-EKF, or on other reduced complexity variants of BONG that employ DLR approximations. We summarize the online training process in Algorithm 1, which fuses module-wise parallel filtering with pipelined execution.

Algorithm 1 Modular online Bayesian training at time t

Require: Pilot queue $\{(\mathbf{r}_{t-i}, \mathbf{b}_{t-i})\}_{i=0}^{Q-1}$
Require: Variational parameters $\{\psi_{t-1}^{(k,q)}\}_{k,q=1,1}^{K,Q}$
Require: Predictions from previous step $\{\ell_{t-1}^{(q)}\}_{q=1}^{Q-1}$

- 1: $\ell_{t-1}^{(0)} \leftarrow [0.5, \dots, 0.5]^\top$
- 2: **for** $q = 1, \dots, Q$ **do**
- 3: $\tau \leftarrow t - (q - 1)$
- 4: $\hat{\mathbf{x}}_t^{(q)} \leftarrow \left[\Re \{ \mathbf{r}_\tau^\top \}, \Im \{ \mathbf{r}_\tau^\top \}, \left(\ell_{t-1}^{(q-1)} \right)^\top \right]^\top$
- 5: **for all** $k = 1, \dots, K$ **do**
- 6: Predict: $\psi_{t|t-1}^{(k,q)} \leftarrow \text{predict}(\psi_{t-1}^{(k,q)})$
- 7: Update: $\psi_t^{(k,q)} \leftarrow \text{update}(\psi_{t|t-1}^{(k,q)}, \hat{\mathbf{x}}_t^{(q)}, \mathbf{b}_\tau^{(k)})$
- 8: Inference: $\ell_t^{(k,q)} \leftarrow \mathbf{h}_{\mu_t^{(k,q)}}(\hat{\mathbf{x}}_t^{(q)})$
- 9: **end for**
- 10: $\ell_t^{(q)\top} \leftarrow [(\ell_t^{(1,q)})^\top, \dots, (\ell_t^{(K,q)})^\top]^\top$
- 11: **end for**

Ensure: Updated variational parameters $\{\psi_t^{(k,q)}\}_{k,q=1,1}^{K,Q}$

In terms of overhead, it is noted that each module-wise operation, i.e., the *Predict-Update-Inference* steps in Algorithm 1, requires two forward passes of $\hat{\mathbf{x}}_t^{(q)}$ through the module, and one backward pass for gradient computation. Assuming the more-computationally complex CM-EKF is employed, the complexity order of these steps is of order $\mathcal{O}(KBP^2)$ each. However, as opposed to the usage of such methods for training an end-to-end DNN as detailed in Subsection III-B, here P represents the parameters of a compact module, which can be relatively small (e.g., $P < 1000$ in our numerical study), as opposed to the overall DNN. For comparison, training DeepSIC with Q iterations and K users via end-to-end CM-EKF would result in complexity order of $\mathcal{O}(BQ^2K^3P^2)$, while module-wise learning¹ requires order of $\mathcal{O}(BQKP^2)$.

¹Here, the per-module observation dimension is B , so that the local Jacobian and noise covariance satisfy $\mathbf{H}_t \in \mathbb{R}^{B \times P}$ and $\mathbf{R}_t \in \mathbb{R}^{B \times B}$, where P is the number of trainable parameters in the corresponding module.

Moreover, it is emphasized that our pipelining of the modular architectures results in the *Predict-Update-Inference* steps in Algorithm 1 being independent with each other, and can thus be computed in parallel. Accordingly, given simple parallel computation capabilities, the overhead in applying Algorithm 1 using CM-EKF can be made to be of an order of $\mathcal{O}(BP^2)$, where P here is the number of module parameters (i.e., $\times QK$ reduction due to parallelization). When also combining DLR approximations, its complexity reduces even further to a linear order of $\mathcal{O}(BP)$.

D. Discussion

The proposed modular online Bayesian learning framework addresses the three main challenges introduced in Subsection II-C, namely, non-stationarity, latency/complexity, and nonlinearity (Challenges C1–C3). By introducing modeling assumptions in a gradual and principled manner, our design enables efficient and rapid single-step online learning. Specifically, the modular architecture decomposes the detection task into compact and interpretable sub-networks, reducing the scale of adaptation. The Bayesian SSM in parameter space captures smooth temporal dynamics of the receiver weights, naturally accommodating non-stationarity, and supports treating received pilots as streaming data. Posterior modeling with recursive Bayesian filtering allows each update to be carried out with minimal overhead, offering computationally tractable real-time adaptation in dynamic environments. Accordingly, the Gaussian evolution model in parameter space and the use of local linearizations are modeling choices adopted to enable tractable and low-latency Bayesian filtering. While the reliability of these assumptions depends on the nature of channel variations, and may be less faithful in abrupt environmental changes, such as sudden signal blockages or handovers, it is numerically shown in Section IV that the resulting methodology still leads to rapid and efficient online adaptation.

Modeling the receiver as a Bayesian neural network allows us to associate a distribution with the network parameters, whose covariance naturally quantifies the uncertainty in the current parameter estimate. Quantifying parameter space uncertainty enables the principled combination of new pilot observations with the previously learned model through recursive Bayesian filtering [46]. *Parameter uncertainty* can be converted into *predictive uncertainty* at the output of the receiver via ensembling of models sampled from the parameter space distribution [33] or via distribution methods [47]. Therefore, the parameter uncertainty maintained by the Bayesian update rules serves exclusively as a modeling device that enables efficient online learning, supporting Bayesian filtering in dynamic communication environments.

At the core of our methodology lies the departure from conventional SGD-based training, which dominates deep learning but requires multiple iterations. By casting online learning as a tracking problem, we enable the use of recursive Bayesian filters as training algorithms. While nonlinear Kalman filtering methods have been considered for adapting small neural networks over 25 years ago [55], their usage for learning was largely abandoned as modern machine learning shifted towards extremely large-scale models, where adaptation was

only feasible through stochastic gradient based optimization. Recent attempts to reintroduce restricted Kalman-style learning into deep models, e.g., [56], have primarily targeted offline settings. In contrast, our approach fuses Bayesian filtering with modular architectures, where each module can be individually evaluated and updated, thereby making single-step Bayesian learning not only possible but also computationally efficient.

Our framework gives rise to several extensions for future research. One direction is to explore additional Bayesian online learning formulations that explicitly assess uncertainty through more advanced metrics, such as coverage-based measures [57], which could provide further insight into the reliability of learned parameter distributions in dynamic environments. A related direction is the investigation of ensemble-based Bayesian tracking methods, such as the ensemble Kalman filter and its stochastic variants [57], [58]. While these approaches can provide improved posterior approximations and enhanced uncertainty quantification, their direct application to deep receivers is challenged by the need to maintain and process multiple model instances, leading to increased latency and computational overhead. Developing lightweight ensemble or hybrid filtering schemes that retain the benefits of sampling-based uncertainty representations while meeting the stringent real-time constraints of communication systems is therefore an interesting avenue for future work. Another promising direction is the reduction of the dimensionality of the tracked parameter space by learning lower-dimensional latent representations of the receiver parameters, thereby enabling Bayesian tracking in a compact latent space [20]. Finally, one may consider alternative Bayesian tracking formulations that relax the Gaussian assumptions adopted in this work and allow for non-Gaussian parameter dynamics and unknown SSM parameters, drawing inspiration from recent advances in Bayesian inference for stochastic dynamic systems [59], [60].

IV. EXPERIMENTAL STUDY

This section numerically evaluates the online learning methodologies² proposed in Section III. Based on the common experimental setup detailed in Subsection IV-A, we evaluate four aspects of our methodology: (i) Before assessing communication performance, we commence in Subsection IV-B with comparing the average online learning time of the different algorithms, to understand the real-time feasibility of each method; (ii) In Subsection IV-C we use a synthetic single user channel with a constant achievable rate to demonstrate the ability of single-step Bayesian adaptation to rapidly learn a near-optimal parameterization for a neural receiver, without any domain knowledge, and steadily track as the channel distribution changes in time; (iii) In Subsection IV-D we showcase the advantages of modular architectures for online training on multi-user linear MIMO channels; (iv) Finally, in Subsection IV-E we compare the performance of different algorithms that learn from streaming data, on both linear and nonlinear multi-user MIMO scenarios.

A. Setup

1) *Receiver Architectures:* We utilize three different deep receivers. Our main DNN-aided modular receiver architecture is

²Our code and all hyper-parameters used in this study can be found online at <https://github.com/gusakovy/adaptive-deep-receivers>.

based on DeepSIC [23], with each module comprised of a compact two-layer multi-layer perceptron (MLP). For a non-modular DNN-based receiver we employ a ResNet-type architecture following [13]. For the single user setting (Subsection IV-C), we use a fully-connected network with two layers.

2) *Learning Algorithms*: Since the main objective of this work is to study rapid online learning for deep receivers, the numerical comparisons focus on different training algorithms applied to the same receiver architecture, focusing on the impact of the learning methodology itself. Accordingly, we train the aforementioned architectures using the following online learning algorithms:

- **SGD-I-J** - applies I epochs of mini-batch SGD with a batch size of J on the cross entropy loss of a frequentist (non-Bayesian) model.
- **GD-I** - applies I iterations of gradient descent (GD) (one sample at a time) [61], starting from the previous θ_{t-1} , using a frequentist model based on the cross-entropy loss.
- **BBB-I** Bayes-by-backprop [46] adapted to online learning. For every sample, the update step applies I GD iterations on the online ELBO loss,

$$\mathcal{L}_t(\psi_t) = -\mathbb{E}_{\mathbf{b}_t \sim q_{\psi_t}} [\log p(\mathbf{b}_t | \mathbf{r}_t, \theta_t)] + D_{\text{KL}}(q_{\psi_t}(\theta_t) \parallel q_{\psi_{t|t-1}}(\theta_t)), \quad (34)$$

i.e., for each iteration $i = 1, \dots, I$, set

$$\psi_t^{(i)} = \psi_t^{(i-1)} - \alpha_t \nabla_{\psi_t} \mathcal{L}_t(\psi_t^{(i-1)}), \quad (35)$$

where α_t is the learning rate. The iterations are initialized with $\psi_t^{(0)} = \psi_{t|t-1}$, and the expectation in (34) is approximated by linearization, which we found to be consistently more efficient and faster than the commonly used sample mean.

- **VD-EKF/Lo-Fi/CM-EKF** - BONG implementation based on linearized Gaussian approximation (A1). This leads to different EKF variants, depending on whether DLR (A3) is assumed: VD-EKF ($R = 0$) [18]; Lo-Fi ($0 < R \ll P$) [54]; and CM-EKF ($R = P$) [50].
- **BONG-EF** - The BONG algorithm detailed in Subsection III-B using the EF approach (A2).

Among the compared algorithms, only SGD does not use streaming data, meaning that it processes in batches and is allowed to look back at past samples to perform multiple epochs, while the other algorithms process one sample at a time. It is also noted that the compared learning algorithms include methods that train frequentist DNN parameters (SGD and GD) as well as ones that train the distribution parameters of BNNs. However, for fair comparison, once learning is concluded, performance is compared for all approaches using frequentist inference, which for the BNNs implies using the mean of the variational parameters as a plug-in approximation for inference.

Unless stated otherwise, for all considered experimental scenarios, the state-space hyperparameters are set to $\gamma = 0.999$ and $\sigma^2 = 10^{-3}$. These values, which are shared across all evaluations, were based on experimental results from [19], and further verified through empirical trials. In practice, σ^2 should be chosen to reflect the underlying channel dynamics, while the chosen value for γ was found to be consistently appropriate, as it mainly ensures the variance of the stochastic process does not diverge.

B. Online Learning Latency

The main goal of our single-step online learning methodology is to enable adapting deep receivers to channel variations with an extremely minor overhead. We next show how each of the ingredients incorporated in our framework (single-step Bayesian learning, approximations A2-A3, modular adaptation) translates into latency reduction. We consider a MIMO setting with $K = 3$ users and $N = 5$ antennas, using a modular (DeepSIC) and a non-modular (ResNet) deep receivers. As our focus is on latency, we compare the learning algorithms that operate in a streaming manner (adapt on each incoming sample), under different forms of DLR limitations R (as A3 can be combined with any Bayesian adaptation method). All models were trained using JAX on the same platform, an Intel i7 12th Gen CPU, i.e., on a standard laptop without dedicated AI accelerators. The resulting average latencies are reported in Table II.

Among the EKF-based methods (A1), VD-EKF, which enforces a diagonal covariance, achieves the lowest latency. Lo-Fi, which uses a diagonal + rank- R covariance structure, introduces a moderate overhead compared to VD-EKF but remains in the same latency regime as GD-10 while being considerably more expressive. The full covariance variant CM-EKF incurs a considerable increase in latency. The EF approximation (A2) reduces the latencies of the BONG algorithm for all covariance types, at the cost of stability and performance reduction, as shown in the following subsections.

The effect of modularity is also evident: DeepSIC consistently achieves sub-millisecond adaptation, whereas the same update rules applied to a dense ResNet quickly become impractical. This highlights that combining modular architectures with single-step Bayesian adaptation is essential to meet real-time constraints. Beyond execution time, an important measure is the memory footprint of the proposed Bayesian adaptation. The primary storage requirement for full-rank methods stems from the covariance matrix associated with each module's parameters, which scales as $\mathcal{O}(P^2)$, (recall that P denotes the number of trainable parameters per module). By utilizing a modular architecture where each detection unit is a relatively small neural network (e.g., with around $P \approx 500$ parameters), the memory requirement is kept to approximately 1 MB per module. This modularity is crucial for scalability: in a non-modular, monolithic receiver architecture, the total number of parameters would be significantly larger, forcing the $\mathcal{O}(P^2)$ storage of the covariance matrix into the gigabyte range, resulting in out-of-memory errors noted in Table II.

Algorithm	Σ_t DLR rank R (A3)	Latency/sample [mSec]	
		DeepSIC	ResNet
GD-I $I = 10$	N/A	0.268	0.528
BBB-I $I = 10$	Full: $R = P$	OOM	OOM
	Diag: $R = 0$	0.644	14.12
BONG + A1 (EKF-type)	Full: $R = P$ (CM-EKF)	2.942	OOM
	DLR: $R = 10$ (Lo-Fi)	0.356	8.117
	Diag: $R = 0$ (VD-EKF)	0.103	4.526
BONG + A2 (EF)	Full: $R = P$	1.712	OOM
	DLR: $R = 10$	0.319	1.443
	Diag: $R = 0$	0.083	1.230

TABLE II
AVERAGE TRAINING ITERATION TIME PER SAMPLE, $K, N = (3, 5)$; OUT OF MEMORY (OOM) DENOTES METHODS THAT COULD NOT BE EXECUTED.

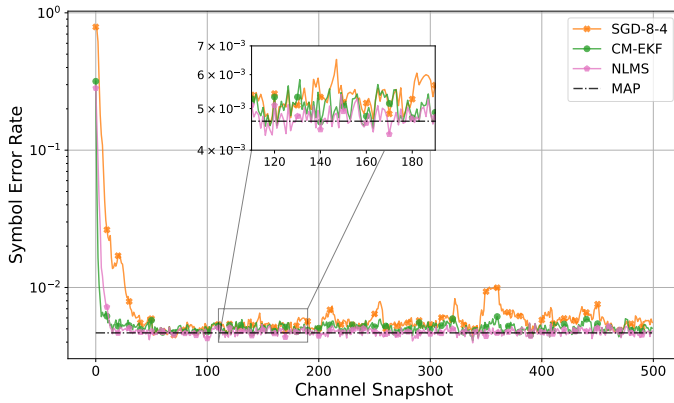


Fig. 5. Symbol error rates, linear rotation channel.

C. Linear Rotation Channel

Here, we consider a simple linear rotation channel with a single user and QPSK symbols. The channel for time t is modeled as a noisy rotation by φ_t radians, i.e.,

$$\mathbf{r}_t = \begin{bmatrix} \cos(\varphi_t) & -\sin(\varphi_t) \\ \sin(\varphi_t) & \cos(\varphi_t) \end{bmatrix} \mathbf{s}_t + \mathbf{u}_t, \quad (36)$$

where $\mathbf{u}_t$ is Gaussian white noise with variance $\sigma_u^2 = 1/16$. The rotation angle increases linearly with parameter $\alpha = 2.5 \cdot 10^{-4}$, i.e., $\varphi_t = 2\pi\alpha t$.

The purpose of this setting is to evaluate our online learning mechanism in a simple setup where model-based receiver algorithms can be applied and approach optimality. Specifically, for the channel in (36), the error rate of the optimal decoder is constant, and can be derived using the maximum *a-posteriori* (MAP) rule. While the MAP rule requires knowledge of the instantaneous channel, it can be efficiently approached in this setting using normalized least mean squares (NLMS) channel estimation followed by equalization and minimum distance decoding. The neural receivers are feed-forward networks with a single hidden layer of dimension 10 followed by a ReLU activation function, and an output layer followed by an elementwise Sigmoid operator that maps to soft bit-level estimates. We compare here CM-EKF-based learning with standard SGD. For both NLMS and SGD, we manually optimized the learning rates.

Fig. 5 illustrates the symbol error rates during 500 channel snapshots, using 16 pilots per snapshot. The average error rate of CM-EKF is lower than that of learning rate optimized SGD performing 8 epochs with batch size 4. Furthermore, it converges faster than both SGD-8-4 and NLMS, taking only 6 snapshots to reach within 0.2% of the optimal symbol error rate, compared to around 12 it takes NLMS and 40 it takes SGD. Training using SGD is also less stable, with a significantly higher peak error when tracking. Fig. 6 visualizes the decision zones of each receiver after 500 blocks. The CM-EKF trained decoder closely matches the MAP decision zones, underlining the ability of the Bayesian receiver to generalize well, while the SGD trained decoder noticeably distorts the decision zones, overfitting to noisy observations.

D. Modular vs. Non-Modular Architectures

We proceed to demonstrate the usefulness of modular architectures for enabling efficient online learning using our Bayesian methodology. To that aim, we compare the modular DeepSIC architecture with a black-box DNN based on a ResNet architecture, known to be suitable for deep receivers [13]. The DNN is comprised of an input layer followed by ReLU activation, 4 residual layers [62], and an output layer followed by an elementwise Sigmoid operator. The input of this ResNet is the received signal $\mathbf{r}_t$, and the output is a vector of soft bit-level estimates for all users, ℓ_t .

The data for this experiment is generated from the QuaDRiGa MIMO geometry-based simulator [40], using the 3GPP line-of-sight configuration, having $K = 3$ single-antenna users in motion transmit to a $N = 5$ antenna receiver. We take 100 consecutive channel snapshots at constant intervals, and 64 symbols are transmitted for each snapshot. The first 4 snapshots are used for synchronization, i.e., $T_{\text{sync}} = 256$, and for the rest transmit 16 pilots per snapshot with the intention of providing sufficient training data. All results are averaged over 10 trials.

Fig. 7 illustrates the average BER vs. signal-to-noise ratio (SNR) for modular and non-modular architectures. Both ResNet and DeepSIC were trained online using SGD-8-4 on pilots from each channel snapshot separately, and BBB-DIAG-10, VD-EKF, Lo-Fi with rank $R = 10$, and CM-EKF on streaming data. The ResNet architecture could not be trained using CM-EKF due to the size of the network ($P = 32,766$). The results highlight the benefits of modular architectures for online adaptation. For the non-modular ResNet receiver, BBB fails to train the network entirely, resulting in consistently poor performance across all SNR values. Training with VD-EKF and SGD-8-4 allows the ResNet to synchronize, but it still struggles to maintain low error rates during the tracking period. Lo-Fi improves the performance in the tracking phase, but it still remains lower than all methods when applied to the modular DeepSIC receiver. In contrast, each module of the modular DeepSIC architecture is a much smaller network ($P = 458$) tasked with decoding a single user, which makes synchronization easier and substantially improves online adaptability. The EKF variants perform markedly better than 10 iterations of either SGD and BBB-DIAG. VD-EKF is the least expressive of the EKF variants but has lower runtime than BBB-DIAG-10. Lo-Fi has a runtime comparable to that of BBB-DIAG-10 with improved error rate, and CM-EKF has the lowest error rates across all SNR values, but is slower than BBB-10.

E. Streaming-Data Algorithms

Having demonstrated the usefulness of combining modular architectures with online Bayesian learning, we proceed to compare algorithms that learn online from streaming data with DeepSIC. Specifically, we consider the Bayesian algorithms BBB-DIAG-10, Lo-Fi with rank $R = 10$, CM-EKF and BONG-EF, as well as the non-Bayesian GD-10 algorithm. All results are averaged over 10 trials.

The data for this experiment is generated from three sources:

- (i) A batch of static narrowband channels generated using the Sionna [38] differentiable link level simulator using the urban microcell (UMi) channel model, having $K = 4$

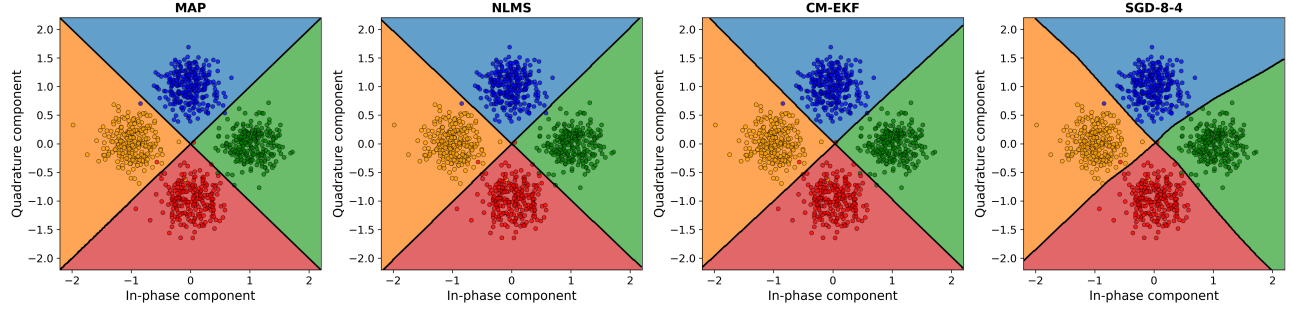


Fig. 6. Decision zones after 500 blocks (45° counter-clockwise rotation) for four decoders - optimal decoding using the MAP rule, channel estimation using NLMS followed by maximum likelihood decoding, and two neural decoders trained using CM-EKF and SGD-8-4.

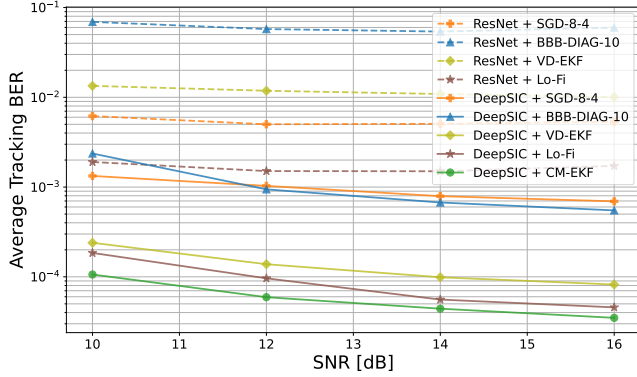


Fig. 7. Online learning for modular vs. non-modular architectures.

single antenna users, randomly scattered within a cell sector, transmit to a single $N = 8$ antenna receiver. We take 4 different channels and use $T_{\text{sync}} = 256$ pilots for synchronization to each of them.

- (ii) The COST2100 [39] geometry-based channel simulator, using the dynamic "indoor closely spaced users" configuration, having $K = 3$ users transmit to a $N = 5$ receiver antenna. We take 300 consecutive channel snapshots at constant intervals, each comprised of 64 symbols, with the first 2 snapshots used for synchronization, i.e., $T_{\text{sync}} = 128$, while the remaining snapshots include only 2 pilots each;
- (iii) A nonlinear channel acquired by applying a nonlinear transformation, and particularly a tanh to a linear channel generated from the QuaDRiGa simulator using a configuration similar to the one used in Subsection IV-D. This operation may represent distortions induced by the receiver acquisition hardware. We take 100 channel snapshots of 64 symbols, with the first 4 used for synchronization, i.e., $T_{\text{sync}} = 256$, while the remaining snapshots include only 16 pilots.

For the Sionna channel synchronization experiment (Fig. 8), we look at synchronization performance for 4 different channels with 256 pilots transmitted for each of them. Since within every window of 256 pilots the channel is constant, we report the BER of the minimum mean square error (MMSE) decoder with full channel state information (CSI) for reference. In addition, we also use model-agnostic meta-learning (MAML) [63] to learn an initialization for quick adaptation to new channels. To do this, we generate additional channels from the same model, and use

meta learning to train an initial parameterization for the DeepSIC receiver, which minimizes the BER during synchronization with GD-10. The learned initialization reduces the BER of GD-10 throughout the entire synchronization period, helping it synchronize faster than GD-10 with random initialization, and BBB-DIAG-10. Even so, the EKF-based methods Lo-Fi and CM-EKF synchronize much faster than the iterative methods, both achieving near optimal error rates using only 256 samples. BONG-EF, on the other hand, synchronizes slower than all other methods, likely attributed to the noisy EF approximation.

For the COST2100 channel experiment (Fig. 9), the CM-EKF outperforms all other methods across all SNR values, with BONG-EF coming in as a close second at high SNR. These full-covariance single-step approaches are consistently able to leverage the limited observations to more effectively adapt the receiver parameters to the varying channel conditions. However, BONG-EF requires significantly more time to synchronize with the channel, due to the inherent instability introduced by using the EF approximation instead of the Hessian. By contrast, Lo-Fi synchronizes faster than any other method, thanks to its lighter yet still expressive parameterization. When tracking, Lo-Fi does fall short of CM-EKF and BONG-EF, but runs a lot faster. BBB-DIAG-10 and GD-10 show comparable performance: both manage to adapt during the synchronization period, but fail to sustain low error rates in the subsequent tracking phase. This degradation can be attributed to the noisy parameter updates that occur when operating with scarce data.

In the nonlinear scenario (Fig. 10), the channel conditions are substantially harsher, demanding highly accurate parameter updates. In this setting, CM-EKF decisively outperforms all other methods across all SNR values by several standard deviations. It not only synchronizes more rapidly than most of the alternatives except Lo-Fi, but it also consistently maintains superior performance throughout the entire tracking period. In comparison, Lo-Fi exhibits the second-best error rates during tracking across all SNR values, further underlining the trade-off between complexity and tracking performance. BONG-EF exhibits slower synchronization and only marginally surpasses the performance of GD-10. Meanwhile, BBB-DIAG-10 struggles to deliver stable tracking, failing to sustain reliable performance under the severe nonlinear conditions.

We conclude our numerical study by examining the sensitivity of the proposed online learning framework to the state evolution hyperparameters. To that aim, we conduct an additional exper-

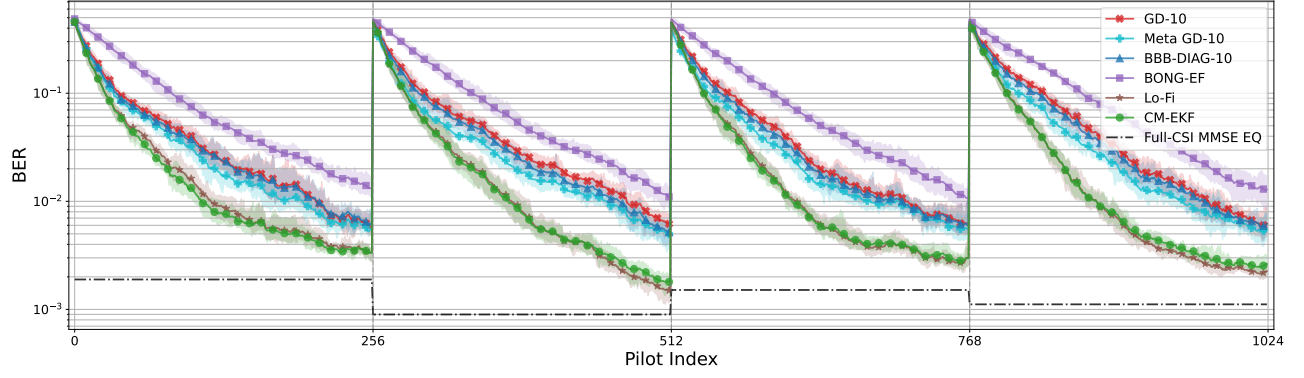


Fig. 8. BER vs. pilot index at SNR=4 dB. Synchronization from streaming data, with four different Sionna narrowband linear channels.

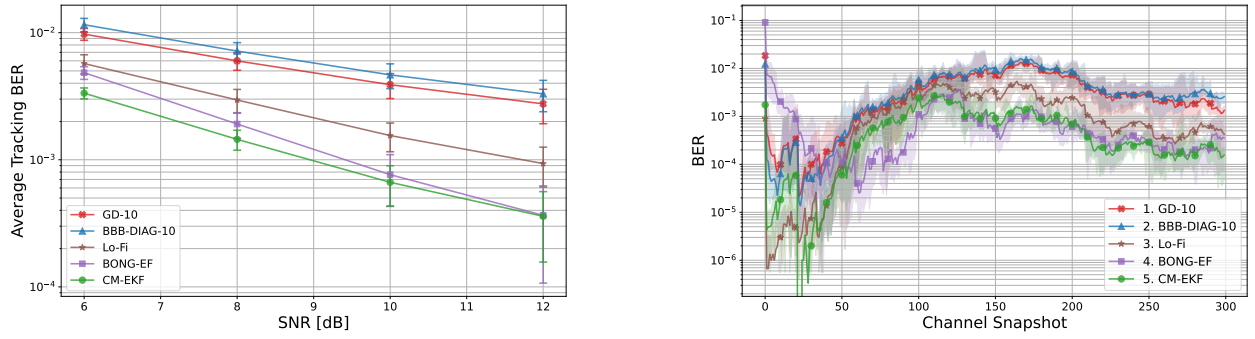


Fig. 9. Learning from streaming data, COST2100 channel. Left: Average BER vs. SNR; Right: BER vs channel snapshot at SNR= 10 dB.

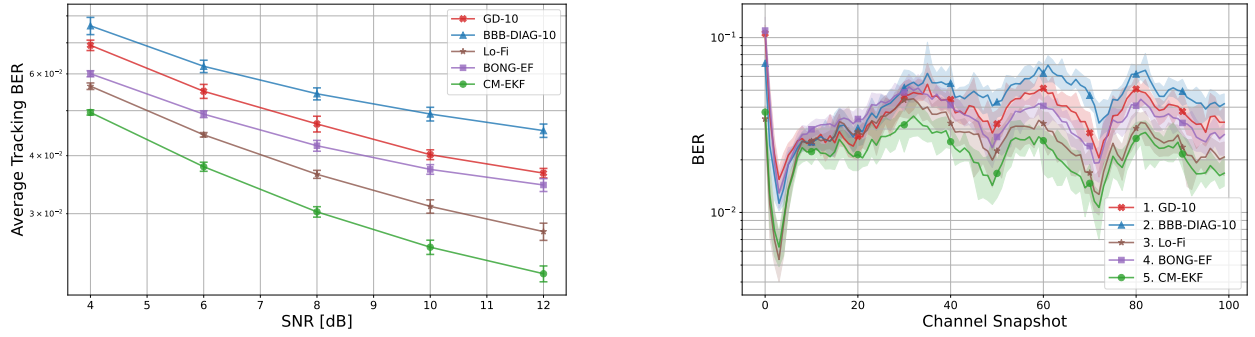


Fig. 10. Learning from streaming data, nonlinear QuaDRiGa channel. Left: Average BER vs. SNR; Right: BER vs channel snapshot at SNR= 12 dB.

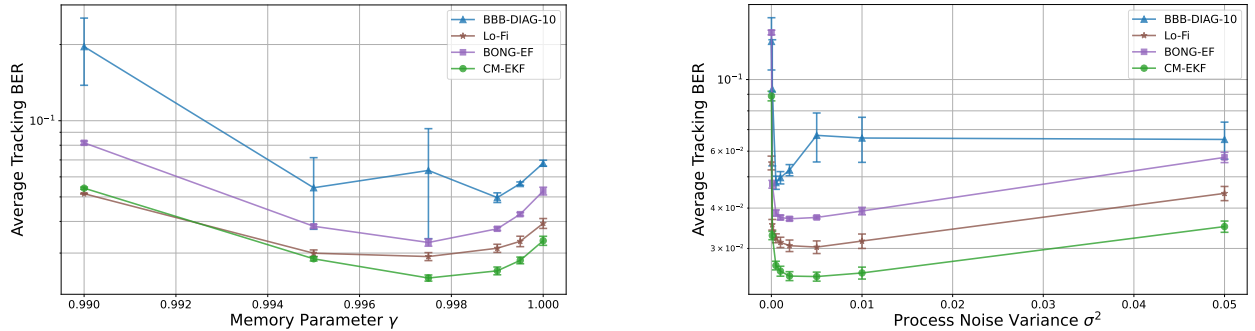


Fig. 11. Sensitivity to state evolution parameters, nonlinear QuaDRiGa channel at SNR= 12 dB. Left: Average BER vs. γ ; Right: Average BER vs. σ^2 .

iment using the nonlinear QuaDRiGa scenario at SNR= 12 dB, and evaluate the average BER achieved by all the streaming-data algorithms while varying the memory parameter γ and the process noise variance σ^2 governing the state evolution model. The results, shown in Fig. 11, indicate that similar ranges of hyperparameters γ and σ^2 provide favorable performance across all the considered algorithms. This suggests that these parameters primarily reflect the dynamics of the underlying channel evolution, rather than the specific choice of the online learning algorithm.

Overall, the results highlight the interplay between the different complexity–performance mechanisms discussed in Subsection III-B. First, the structure imposed on the covariance matrix determines the balance between expressiveness and computational cost. Full-covariance tracking with CM-EKF consistently yields the most reliable adaptation, particularly in challenging or nonlinear scenarios, while the DLR and diagonal approximations (A3) provide a favorable compromise between latency and tracking accuracy, often synchronizing the fastest; Second, replacing the Hessian with the empirical Fisher approximation (A2) further reduces computational overhead, but may introduce noisy updates that slow synchronization and degrade stability compared to using EKF-type methods (A1). Accordingly, practitioners seeking the highest robustness should favor EKF-based variants (with CM-EKF offering the best performance and Lo-Fi providing a fast and stable alternative), whereas the empirical Fisher variant is most appropriate in settings where minimizing online learning latency is the primary concern and some loss in stability can be tolerated.

V. CONCLUSIONS

In this work, we proposed a modular online Bayesian learning framework for deep receivers operating over time-varying channels. By casting online learning as a state-space tracking problem and adopting recursive Bayesian updates, we demonstrated that reliable one-shot adaptation can be achieved, replacing multi-epoch training with single-step updates. Furthermore, by leveraging modular receiver architectures such as DeepSIC, we showed how module-wise Bayesian adaptation from streaming pilots enables parallelization and pipelining, thereby substantially reducing both complexity and latency.

REFERENCES

- [1] Y. Gusakov, O. Simeone, T. Rountenberg, and N. Shlezinger, “Rapid online Bayesian learning for deep receivers,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.
- [2] C. Huang, S. Hu, G. C. Alexandropoulos, A. Zappone, C. Yuen, R. Zhang, M. Di Renzo, and M. Debbah, “Holographic MIMO surfaces for 6G wireless networks: Opportunities, challenges, and trends,” *IEEE Commun. Mag.*, vol. 27, no. 5, pp. 118–125, 2020.
- [3] Y. Liu, X. Liu, X. Mu, T. Hou, J. Xu, M. Di Renzo, and N. Al-Dhahir, “Reconfigurable intelligent surfaces: Principles and opportunities,” *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1546–1577, 2021.
- [4] N. Shlezinger, G. C. Alexandropoulos, M. F. Imani, Y. C. Eldar, and D. R. Smith, “Dynamic metasurface antennas for 6G extreme massive MIMO communications,” *IEEE Wireless Commun.*, vol. 28, no. 2, pp. 106–113, 2021.
- [5] N. Lang, Y. Gabay, N. Shlezinger, T. Rountenberg, Y. Ghasempour, G. C. Alexandropoulos, and Y. C. Eldar, “Wideband THz multi-user downlink communications with leaky wave antennas,” *IEEE Trans. Wireless Commun.*, vol. 25, pp. 6921–6937, 2025.
- [6] L. Dai, R. Jiao, F. Adachi, H. V. Poor, and L. Hanzo, “Deep learning for wireless communications: An emerging interdisciplinary paradigm,” *IEEE Wireless Commun.*, vol. 27, no. 4, pp. 133–139, 2020.
- [7] T. O’Shea and J. Hoydis, “An introduction to deep learning for the physical layer,” *IEEE Trans. on Cogn. Commun. Netw.*, vol. 3, no. 4, pp. 563–575, 2017.
- [8] N. Shlezinger, M. Ma, O. Lavi, N. T. Nguyen, Y. C. Eldar, and M. Juntti, “Artificial intelligence-empowered hybrid multiple-input/multiple-output beamforming: Learning to optimize for high-throughput scalable MIMO,” *IEEE Veh. Technol. Mag.*, vol. 19, no. 3, pp. 58–67, 2024.
- [9] W. Tong and G. Y. Li, “Nine challenges in artificial intelligence and wireless communications for 6G,” *IEEE Wireless Commun.*, vol. 29, no. 4, pp. 140–145, 2022.
- [10] T. Raviv, S. Park, O. Simeone, Y. C. Eldar, and N. Shlezinger, “Adaptive and flexible model-based AI for deep receivers in dynamic channels,” *IEEE Wireless Commun.*, vol. 31, no. 4, pp. 163–169, 2024.
- [11] J. Xia, D. Deng, and D. Fan, “A note on implementation methodologies of deep learning-based signal detection for conventional MIMO transmitters,” *IEEE Trans. Broadcast.*, vol. 66, no. 3, pp. 744–745, 2020.
- [12] T. Raviv, A. Goldman, O. Vayner, Y. Be’ery, and N. Shlezinger, “CRC-aided learned ensembles of belief-propagation polar decoders,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 8856–8860.
- [13] M. Honkala, D. Korpi, and J. M. Huttunen, “DeepRx: Fully convolutional deep learning receiver,” *IEEE Trans. Wireless Commun.*, vol. 20, no. 6, pp. 3925–3940, 2021.
- [14] M. Goutay, F. A. Aoudia, J. Hoydis, and J.-M. Gorce, “Machine learning for MU-MIMO receive processing in OFDM systems,” *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2318–2332, 2021.
- [15] G. Liu, Z. Hu, L. Wang, H. Zhang, J. Xue, and M. Matthaiou, “A hypernetwork based framework for non-stationary channel prediction,” *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 8338–8351, 2024.
- [16] T. Raviv and N. Shlezinger, “Modular hypernetworks for scalable and adaptive deep MIMO receivers,” *IEEE Open Journal of Signal Processing*, vol. 6, pp. 256–265, 2025.
- [17] M. Zecchin, K. Yu, and O. Simeone, “In-context learning for MIMO equalization using transformer-based sequence models,” in *IEEE International Conference on Communications Workshops (ICC Workshops)*, 2024, pp. 1573–1578.
- [18] P. G. Chang, K. P. Murphy, and M. Jones, “On diagonal approximations to the extended Kalman filter for online training of Bayesian neural networks,” in *Continual Lifelong Learning Workshop at ACML 2022*, 2022.
- [19] M. Jones, P. Chang, and K. Murphy, “Bayesian online natural gradient (BONG),” *Advances in Neural Information Processing Systems*, vol. 37, pp. 131 104–131 153, 2024.
- [20] G. Duran-Martin, L. Sánchez-Betancourt, A. Y. Shestopaloff, and K. Murphy, “A unifying framework for generalised Bayesian online learning in non-stationary environments,” *Transactions on Machine Learning Research*, 2025.
- [21] J. Durbin and S. J. Koopman, *Time series analysis by state space methods*. OUP Oxford, 2012, vol. 38.
- [22] N. Shlezinger, N. Farsad, Y. C. Eldar, and A. J. Goldsmith, “ViterbiNet: A deep learning based Viterbi algorithm for symbol detection,” *IEEE Trans. Wireless Commun.*, vol. 19, no. 5, pp. 3319–3331, 2020.
- [23] N. Shlezinger, R. Fu, and Y. C. Eldar, “DeepSIC: Deep soft interference cancellation for multiuser MIMO detection,” *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1349–1362, 2021.
- [24] N. Shlezinger and Y. C. Eldar, “Model-based deep learning,” *Foundations and Trends® in Signal Processing*, vol. 17, no. 4, pp. 291–416, 2023.
- [25] S. Park, H. Jang, O. Simeone, and J. Kang, “Learning to demodulate from few pilots via offline and online meta-learning,” *IEEE Trans. Signal Process.*, vol. 69, pp. 226 – 239, 2020.
- [26] T. Raviv, S. Park, O. Simeone, Y. C. Eldar, and N. Shlezinger, “Online meta-learning for hybrid model-based deep receivers,” *IEEE Trans. Wireless Commun.*, vol. 22, no. 10, pp. 6415–6431, 2023.
- [27] N. Uzlaner, T. Raviv, N. Shlezinger, and K. Todros, “Asynchronous online adaptation via modular drift detection for deep receivers,” *IEEE Trans. Wireless Commun.*, vol. 24, no. 5, pp. 4454–4468, 2025.
- [28] F. A. Aoudia and J. Hoydis, “End-to-end learning for OFDM: From neural receivers to pilotless communication,” *IEEE Trans. Wireless Commun.*, vol. 21, no. 2, pp. 1049–1063, 2021.
- [29] M. B. Fischer, S. Dörner, S. Cammerer, T. Shimizu, H. Lu, and S. Ten Brink, “Adaptive neural network-based OFDM receivers,” in *IEEE Signal Processing Advances in Wireless Communications (SPAWC)*, 2022.
- [30] R. Finish, Y. Cohen, T. Raviv, and N. Shlezinger, “Symbol-level online channel tracking for deep receivers,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 8897–8901.
- [31] L. Huang, W. Pan, Y. Zhang, L. Qian, N. Gao, and Y. Wu, “Data augmentation for deep learning-based radio modulation classification,” *IEEE Access*, vol. 8, pp. 1498–1506, 2019.

- [32] T. Raviv and N. Shlezinger, "Data augmentation for deep receivers," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 8259–8274, 2023.
- [33] O. Simeone, *Machine Learning for Engineers*. Cambridge University Press, 2022.
- [34] K. M. Cohen, S. Park, O. Simeone, and S. Shamai, "Bayesian active meta-learning for reliable and efficient AI-based demodulation," *IEEE Trans. Signal Process.*, vol. 70, pp. 5366–5380, 2022.
- [35] M. Zecchin, S. Park, O. Simeone, M. Kountouris, and D. Gesbert, "Robust Bayesian learning for reliable wireless AI: Framework and applications," *IEEE Trans. on Cogn. Commun. Netw.*, vol. 9, no. 4, pp. 897–912, 2023.
- [36] T. Raviv, S. Park, O. Simeone, and N. Shlezinger, "Uncertainty-aware and reliable neural MIMO receivers via modular Bayesian deep learning," *IEEE Trans. Veh. Technol.*, vol. 74, no. 11, pp. 17 637–17 651, 2025.
- [37] S. Farquhar and Y. Gal, "A unifying Bayesian view of continual learning," *arXiv preprint arXiv:1902.06494*, 2019.
- [38] J. Hoydis, S. Cammerer, F. Ait Aoudia, M. Nimier-David, L. Maggi, G. Marcus, A. Vem, and A. Keller, "Sionna: An open-source library for research on communication systems," 2022, <https://nvlabs.github.io/sionna/>.
- [39] L. Liu, C. Oestges, J. Poutanen, K. Haneda, P. Vainikainen, F. Quitin, F. Tufvesson, and P. De Doncker, "The COST 2100 MIMO channel model," *IEEE Wireless Commun.*, vol. 19, no. 6, pp. 92–99, 2012.
- [40] S. Jäckel, L. Raschkowski, K. Börner, and L. Thiele, "Quadriga: A 3-D multi-cell channel model with time evolution for enabling virtual field trials," *IEEE Trans. Antennas Propag.*, vol. 62, no. 6, pp. 3242–3256, 2014.
- [41] B. Hassibi and B. M. Hochwald, "How much training is needed in multiple-antenna wireless links?" *IEEE Trans. Inf. Theory*, vol. 49, no. 4, pp. 951–963, 2003.
- [42] K. T. Truong and R. W. Heath, "Effects of channel aging in massive mimo systems," *Journal of Communications and Networks*, vol. 15, no. 4, pp. 338–351, 2013.
- [43] A. G. Wilson and P. Izmailov, "Bayesian deep learning and a probabilistic perspective of generalization," *Advances in Neural Information Processing Systems*, vol. 33, pp. 4697–4708, 2020.
- [44] V. Fortuin, "Priors in Bayesian deep learning: A review," *International Statistical Review*, vol. 90, no. 3, pp. 563–591, 2022.
- [45] Y. Gal, J. Hron, and A. Kendall, "Concrete dropout," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [46] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, "Weight uncertainty in neural network," in *International Conference on Machine Learning*. PMLR, 2015, pp. 1613–1622.
- [47] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher *et al.*, "A survey of uncertainty in deep neural networks," *Artificial Intelligence Review*, vol. 56, no. Suppl 1, pp. 1513–1589, 2023.
- [48] N. Shlezinger, Y. C. Eldar, and S. P. Boyd, "Model-based deep learning: On the intersection of deep learning and optimization," *IEEE Access*, vol. 10, pp. 115 384–115 398, 2022.
- [49] W.-J. Choi, K.-W. Cheong, and J. M. Cioffi, "Iterative soft interference cancellation for multiple antenna systems," in *Proc. IEEE WCNC*, 2000.
- [50] F. Tronarp, A. F. García-Fernández, and S. Särkkä, "Iterative filtering and smoothing in nonlinear and non-Gaussian systems using conditional moments," *IEEE Signal Processing Letters*, vol. 25, no. 3, pp. 408–412, 2018.
- [51] A. Papoulis and S. U. Pillai, *Probability, Random Variables, and Stochastic Processes*, 4th ed. New York: McGraw-Hill, 2002, see p. 127.
- [52] J. Martens, "New insights and perspectives on the natural gradient method," *J. Mach. Learn. Res.*, vol. 21, no. 1, Jan. 2020.
- [53] A. Mishkin, F. Kunstner, D. Nielsen, M. Schmidt, and M. E. Khan, "SLANG: Fast structured covariance approximations for Bayesian deep learning with natural gradient," in *Advances in Neural Information Processing Systems*, 2018.
- [54] P. G. Chang, G. Durán-Martín, A. Y. Shestopaloff, M. Jones, and K. Murphy, "Low-rank extended Kalman filtering for online learning of neural networks from streaming data," *arXiv preprint arXiv:2305.19535*, 2023.
- [55] E. A. Wan and R. Van Der Merwe, "The unscented Kalman filter for nonlinear estimation," in *IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium*, 2000, pp. 153–158.
- [56] Z. Xia, A. Davtyan, and P. Favaro, "KOALA++: Efficient Kalman-based optimization of neural networks with gradient-covariance products," in *Advances in Neural Information Processing Systems*, 2025.
- [57] P. Zhang, Q. Song, and F. Liang, "A Langevinized ensemble Kalman filter for large-scale dynamic learning," *Statistica Sinica*, vol. 34, pp. 1071–1091, 2024.
- [58] G. Evensen, "The ensemble Kalman filter: Theoretical formulation and practical implementation," *Ocean dynamics*, vol. 53, no. 4, pp. 343–367, 2003.
- [59] P. Zhang, T. Dong, and F. Liang, "An extended Langevinized ensemble Kalman filter for non-Gaussian dynamic systems," *Computational statistics*, vol. 39, no. 6, pp. 3347–3372, 2024.
- [60] T. Dong, P. Zhang, and F. Liang, "A stochastic approximation-Langevinized ensemble Kalman filter algorithm for state space models with unknown parameters," *Journal of Computational and Graphical Statistics*, vol. 32, no. 2, pp. 448–469, 2023.
- [61] M. Zinkevich, "Online convex programming and generalized infinitesimal gradient ascent," in *International Conference on International Conference on Machine Learning*, 2003, p. 928–935.
- [62] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [63] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International Conference on Machine Learning*, 2017, p. 1126–1135.



Yakov Gusakov received his dual B.Sc. in Electrical Engineering and Mathematics (with highest honors) in 2024, and his M.Sc. in Electrical Engineering in 2025, both from Ben-Gurion University of the Negev, Be'er Sheva, Israel. His M.Sc. research, conducted under the supervision of Prof. Tirza Routtenberg and Dr. Nir Shlezinger, focuses on Bayesian online learning for wireless deep receiver design.

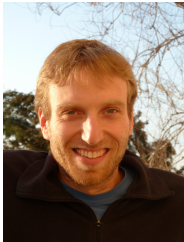


Osvaldo Simeone is the Professor of Information Engineering at Northeastern University London, where he co-directs the Institute for Intelligent Networked Systems (INSI). He is also a visiting Professor with the Connectivity Section within the Department of Electronic Systems at Aalborg University. Previously, he was with the Department of Engineering of King's College London from 2017 to 2025, and with the Electrical and Computer Engineering (ECE) Department at New Jersey Institute of Technology (NJIT) from 2006 to 2017. He received an M.Sc. degree (with honors) and a Ph.D. degree in information engineering from Politecnico di Milano, Milan, Italy, in 2001 and 2005, respectively. His research interests include information theory, machine learning, wireless communications, neuromorphic computing, and quantum machine learning. Dr. Simeone is a co-recipient of the 2025 IEEE ICC Best Paper Award, the 2022 IEEE Communications Society Outstanding Paper Award, the 2021 IEEE Vehicular Technology Society Jack Neubauer Memorial Award, the 2019 IEEE Communication Society Best Tutorial Paper Award, the 2018 IEEE Signal Processing Best Paper Award, the 2017 JCN Best Paper Award, the 2015 IEEE Communication Society Best Tutorial Paper Award and of the Best Paper Awards of IEEE SPAWC 2007 and IEEE WRECOM 2007. He was awarded an Advanced grant by the European Research Council (ERC) in 2025, an Open Fellowship by the EPSRC in 2022, and a Consolidator grant by the ERC in 2016. His research has been also supported by the U.S. National Science Foundation (NSF), the European Commission, the European Research Council (ERC), the Engineering & Physical Sciences Research Council (EPSRC), the Vienna Science and Technology Fund, the European Space Agency, as well as by a number of industrial collaborations including with Intel Labs and InterDigital. He was the Chair of the Signal Processing for Communications and Networking Technical Committee of the IEEE Signal Processing Society in 2022, as well as of the UK & Ireland Chapter of the IEEE Information Theory Society from 2017 to 2022. He was a Distinguished Lecturer of the IEEE Communications Society in 2021 and 2022, and he was a Distinguished Lecturer of the IEEE Information Theory Society in 2017 and 2018. Prof. Simeone is the author of the textbooks "Machine Learning for Engineers" and "Classical and Quantum Information Theory" published by Cambridge University Press, four monographs, two edited books, and more than 200 research journal and magazine papers. He is a Fellow of the IET and IEEE.



Tirza Routtenberg (Senior Member, IEEE) is a Professor in the School of Electrical and Computer Engineering at Ben-Gurion University of the Negev, Israel. She received the B.Sc. degree in biomedical engineering from the Technion–Israel Institute of Technology, Haifa, Israel, in 2005, and the M.Sc. and Ph.D. degrees in electrical and computer engineering from Ben-Gurion University of the Negev, Israel, in 2007 and 2012, respectively. From 2012 to 2014, she was a Postdoctoral Fellow with the School of Electrical and Computer Engineering, Cornell University, Ithaca, NY,

USA. Since 2014, she has been with Ben-Gurion University of the Negev. In 2022–2023, she was a William R. Kenan, Jr. Visiting Professor for Distinguished Teaching at Princeton University, Princeton, NJ, USA. Her research interests include statistical signal processing, estimation and detection theory, signal processing in smart grids, and graph signal processing. She served as an Associate Editor for the *IEEE Transactions on Signal and Information Processing over Networks* (2019–2023) and *IEEE Signal Processing Letters* (2021–2025). She has been a Subject Editor for *Signal Processing* (Elsevier) since 2024. She also served as a Member of the IEEE Signal Processing Society Education Board (2023–2025) and is a member of the IEEE Signal Processing Theory and Methods Technical Committee. She was a recipient or co-recipient of Best Student Paper Awards at ICASSP 2011, CAMSAP 2013, ICASSP 2017, and SSP 2018. She received the Marc Rich Foundation Prize in 2012 and the Toronto Prize for Excellence in Research in 2021.



Nir Shlezinger (M'17-SM'23) is an associate professor in the School of Electrical and Computer Engineering at Ben-Gurion University, Israel. He received his B.Sc., M.Sc., and Ph.D. degrees in 2011, 2013, and 2017, respectively, from Ben-Gurion University, Israel, all in electrical and computer engineering. From 2017 to 2019, he was a postdoctoral researcher at the Technion, and from 2019 to 2020, he was a postdoctoral researcher at the Weizmann Institute of Science, where he was awarded the FGS Prize for his research achievements. He is the recipient of the 2024 IEEE

ComSoc Fred W. Ellersick Award, the 2025 IEEE ComSoc Marconi Prize, and the 2024 Krill Prize for outstanding young researchers. His research interests include communications, information theory, signal processing, and machine learning.